

```
In [1]: # EcoSort AI - Smart Waste Segregation
# Cell 1: Basic imports and setup

import os
import cv2
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

import tensorflow as tf
from tensorflow.keras import layers, models
from tensorflow.keras.preprocessing.image import ImageDataGenerator

from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, confusion_matrix

import datetime

# just to keep results consistent across runs
np.random.seed(42)
tf.random.set_seed(42)

print("TensorFlow version:", tf.__version__)
print("All libraries imported successfully")
```

```
TensorFlow version: 2.20.0
All libraries imported successfully
```

```

In [2]: # Cell 2: Clone dataset and model from GitHub (robust version)

import shutil
import os

# remove any stale clone from a previous run in this session
if os.path.exists('/content/EcoSortAI_repo'):
    shutil.rmtree('/content/EcoSortAI_repo')

if os.path.exists('/content/EcoSortAI'):
    shutil.rmtree('/content/EcoSortAI')

!git clone https://github.com/Maharshi567/EcoSortAI.git
/content/EcoSortAI_repo

import zipfile

repo_files = os.listdir('/content/EcoSortAI_repo')
print("Files found in repo:", repo_files)

zip_files = [f for f in repo_files if f.lower().endswith('.zip')]

if zip_files:
    zip_path = os.path.join('/content/EcoSortAI_repo', zip_files[0])
    with zipfile.ZipFile(zip_path, 'r') as zip_ref:
        zip_ref.extractall('/content/EcoSortAI/dataset_raw')
    print(f"Extracted: {zip_files[0]}")
else:
    print("No zip file found in repo - check your GitHub upload")

CATEGORIES = ['organic', 'recyclable', 'hazardous', 'ewaste']

# auto-detect the actual folder that contains
# organic/recyclable/hazardous/ewaste
# (handles Drive's extra nested folder wrapping)
BASE_DIR = None
for root, dirs, files in os.walk('/content/EcoSortAI/dataset_raw'):
    if all(cat in dirs for cat in CATEGORIES):
        BASE_DIR = root
        break

if BASE_DIR is None:
    print("Could not auto-detect dataset folder structure. Check the zip
contents manually.")
else:
    print(f"Dataset found at: {BASE_DIR}\n")
    for category in CATEGORIES:
        folder_path = os.path.join(BASE_DIR, category)
        total_images = 0
        for r, d, f in os.walk(folder_path):
            total_images += len([x for x in f if
x.lower().endswith(('.jpg', '.jpeg', '.png'))])
        print(f"{category} -> {total_images} images total")

```

```

Cloning into '/content/EcoSortAI_repo'...
remote: Enumerating objects: 4, done. █[K

```

```

remote: Counting objects: 100% (4/4), done.█[K
remote: Compressing objects: 100% (4/4), done.█[K
remote: Total 4 (delta 0), reused 0 (delta 0), pack-reused 0 (from
0)█[K
Receiving objects: 100% (4/4), 11.09 MiB | 11.99 MiB/s, done.
Files found in repo: ['dataset-20260715T101654Z-1-001.zip',
'ecosort_model.h5', '.git']
Extracted: dataset-20260715T101654Z-1-001.zip
Dataset found at: /content/EcoSortAI/dataset_raw/dataset

organic -> 40 images total
recyclable -> 40 images total
hazardous -> 43 images total
ewaste -> 41 images total

```

```

In [6]: # Cell 3: Verify image counts including subfolders

import os

for category in CATEGORIES:
    category_path = os.path.join(BASE_DIR, category)
    total_images = 0
    for root, dirs, files in os.walk(category_path):
        image_files = [f for f in files if f.lower().endswith(('.jpg',
'.jpeg', '.png'))]
        total_images += len(image_files)
    print(f"{category} -> {total_images} images total")

organic -> 0 images total
recyclable -> 0 images total
hazardous -> 0 images total
ewaste -> 0 images total

```

```

In [3]: # Cell 4: Load images and preprocess (without /255 normalization -
        MobileNetV2 needs raw 0-255 pixels)

import cv2
import numpy as np
import os

IMG_SIZE = 224
CATEGORIES = ['organic', 'recyclable', 'hazardous', 'ewaste']
# NOTE: BASE_DIR is NOT redefined here - it uses the value already set
correctly in Cell 2

data = []
labels = []

for label_index, category in enumerate(CATEGORIES):
    category_path = os.path.join(BASE_DIR, category)

    for root, dirs, files in os.walk(category_path):
        for file_name in files:
            if not file_name.lower().endswith(('.jpg', '.jpeg',
            '.png')):
                continue

            img_path = os.path.join(root, file_name)
            img = cv2.imread(img_path)
            if img is None:
                print(f"Skipping unreadable file: {img_path}")
                continue

            img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
            img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
            # NOTE: no /255 here - MobileNetV2's preprocess_input
handles scaling itself

            data.append(img)
            labels.append(label_index)

data = np.array(data, dtype='float32')
labels = np.array(labels)

print(f"Total images loaded: {len(data)}")
print(f>Data shape: {data.shape}")

```

```

Total images loaded: 164
Data shape: (164, 224, 224, 3)

```

```
In [4]: # Cell 5: Split data into train/validation/test and one-hot encode
labels

from sklearn.model_selection import train_test_split
from tensorflow.keras.utils import to_categorical

NUM_CLASSES = 4

# first split: 70% train, 30% temp (which we split again into val/test)
X_train, X_temp, y_train, y_temp = train_test_split(
    data, labels, test_size=0.30, random_state=42, stratify=labels
)

# second split: divide the 30% temp into 15% val, 15% test
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)

# one-hot encode labels for categorical crossentropy
y_train_cat = to_categorical(y_train, NUM_CLASSES)
y_val_cat = to_categorical(y_val, NUM_CLASSES)
y_test_cat = to_categorical(y_test, NUM_CLASSES)

print(f"Training samples: {X_train.shape[0]}")
print(f"Validation samples: {X_val.shape[0]}")
print(f"Test samples: {X_test.shape[0]}")
```

```
Training samples: 114
Validation samples: 25
Test samples: 25
```

```
In [5]: # Cell 6: Transfer learning using MobileNetV2 (pretrained on ImageNet)

from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras.preprocessing.image import ImageDataGenerator
from tensorflow.keras import layers, models

# augmentation for training data only
train_datagen = ImageDataGenerator(
    rotation_range=30,
    width_shift_range=0.2,
    height_shift_range=0.2,
    shear_range=0.15,
    zoom_range=0.2,
    horizontal_flip=True,
    brightness_range=[0.8, 1.2],
    fill_mode='nearest'
)

train_datagen.fit(X_train)

# load MobileNetV2 pretrained on ImageNet, without its final
classification layer
base_model = MobileNetV2(
    input_shape=(224, 224, 3),
    include_top=False,
    weights='imagenet'
)

# freeze the pretrained layers so we don't destroy what it already
learned
base_model.trainable = False

# add our own classification head on top
model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(NUM_CLASSES, activation='softmax')
])

model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	
mobilenetv2_1.00_224 (Functional)	(None, 7, 7, 1280)	2,
global_average_pooling2d (GlobalAveragePooling2D)	(None, 1280)	
dense (Dense)	(None, 128)	
dropout (Dropout)	(None, 128)	

dense_1 (Dense)	(None, 4)
-----------------	-----------

Total params: 2,422,468 (9.24 MB)

Trainable params: 164,484 (642.52 KB)

Non-trainable params: 2,257,984 (8.61 MB)

```

In [6]: # Cell 7: Compile and train the transfer learning model

from tensorflow.keras.callbacks import EarlyStopping

model.compile(
    optimizer='adam',
    loss='categorical_crossentropy',
    metrics=['accuracy']
)

early_stop = EarlyStopping(
    monitor='val_loss',
    patience=5,
    restore_best_weights=True
)

BATCH_SIZE = 16

history = model.fit(
    train_datagen.flow(X_train, y_train_cat, batch_size=BATCH_SIZE),
    validation_data=(X_val, y_val_cat),
    epochs=20,
    callbacks=[early_stop]
)

```

```

Epoch 1/20
1s/step - accuracy: 0.2456 - loss: 1.9972 - val_accuracy: 0.2800 - val_loss: 1.3940
Epoch 2/20
1s/step - accuracy: 0.4211 - loss: 1.5228 - val_accuracy: 0.5600 - val_loss: 1.1704
Epoch 3/20
840ms/step - accuracy: 0.4474 - loss: 1.3533 - val_accuracy: 0.5200 - val_loss: 0.9989
Epoch 4/20
1s/step - accuracy: 0.5263 - loss: 1.1467 - val_accuracy: 0.6800 - val_loss: 0.9446
Epoch 5/20
864ms/step - accuracy: 0.5789 - loss: 1.0063 - val_accuracy: 0.6800 - val_loss: 0.9009
Epoch 6/20
1s/step - accuracy: 0.6579 - loss: 0.9049 - val_accuracy: 0.6400 - val_loss: 0.8943
Epoch 7/20
1s/step - accuracy: 0.5439 - loss: 1.0709 - val_accuracy: 0.5200 - val_loss: 0.8633
Epoch 8/20
810ms/step - accuracy: 0.6404 - loss: 0.8953 - val_accuracy:

```

0.7600 - val_loss: 0.7788
Epoch 9/20
[1m8/8][0m 32m-----[0m37m[0m 1m10s[0m
1s/step - accuracy: 0.6579 - loss: 0.8833 - val_accuracy: 0.6800 -
val_loss: 0.7475
Epoch 10/20
[1m8/8][0m 32m-----[0m37m[0m 1m7s[0m
923ms/step - accuracy: 0.7105 - loss: 0.8365 - val_accuracy:
0.6800 - val_loss: 0.7397
Epoch 11/20
[1m8/8][0m 32m-----[0m37m[0m 1m9s[0m
1s/step - accuracy: 0.6930 - loss: 0.7523 - val_accuracy: 0.6400 -
val_loss: 0.6280
Epoch 12/20
[1m8/8][0m 32m-----[0m37m[0m 1m7s[0m
821ms/step - accuracy: 0.6842 - loss: 0.8093 - val_accuracy:
0.9200 - val_loss: 0.5939
Epoch 13/20
[1m8/8][0m 32m-----[0m37m[0m 1m9s[0m
1s/step - accuracy: 0.6667 - loss: 0.7365 - val_accuracy: 0.7200 -
val_loss: 0.6176
Epoch 14/20
[1m8/8][0m 32m-----[0m37m[0m 1m7s[0m
824ms/step - accuracy: 0.6842 - loss: 0.7128 - val_accuracy:
0.8000 - val_loss: 0.6061
Epoch 15/20
[1m8/8][0m 32m-----[0m37m[0m 1m10s[0m
1s/step - accuracy: 0.7105 - loss: 0.6915 - val_accuracy: 0.7600 -
val_loss: 0.5854
Epoch 16/20
[1m8/8][0m 32m-----[0m37m[0m 1m7s[0m
879ms/step - accuracy: 0.6842 - loss: 0.6096 - val_accuracy:
0.8400 - val_loss: 0.5277
Epoch 17/20
[1m8/8][0m 32m-----[0m37m[0m 1m11s[0m
1s/step - accuracy: 0.7544 - loss: 0.6754 - val_accuracy: 0.8000 -
val_loss: 0.5975
Epoch 18/20
[1m8/8][0m 32m-----[0m37m[0m 1m7s[0m
815ms/step - accuracy: 0.7632 - loss: 0.6059 - val_accuracy:
0.8000 - val_loss: 0.4340
Epoch 19/20
[1m8/8][0m 32m-----[0m37m[0m 1m14s[0m
2s/step - accuracy: 0.7544 - loss: 0.6829 - val_accuracy: 0.8800 -
val_loss: 0.5127
Epoch 20/20
[1m8/8][0m 32m-----[0m37m[0m 1m14s[0m
891ms/step - accuracy: 0.6930 - loss: 0.6990 - val_accuracy:
0.8400 - val_loss: 0.5036

```
In [7]: # Cell 7: Evaluate on test set and save the trained model

test_loss, test_accuracy = model.evaluate(X_test, y_test_cat)
print(f"\nTest Accuracy: {test_accuracy*100:.2f}%")
print(f"Test Loss: {test_loss:.4f}")

# save the trained model locally in this Colab session
MODEL_SAVE_PATH = '/content/EcoSortAI/ecosort_model.h5'
model.save(MODEL_SAVE_PATH)
print(f"\nModel saved to: {MODEL_SAVE_PATH}")
```

1m1/1m 32m — 0m 37m 0m 1m1s 0m
1s/step - accuracy: 0.6000 - loss: 1.0054

WARNING:absl:You are saving your model as an HDF5 file via
`model.save()` or `keras.saving.save\_model(model)`. This file
format is considered legacy. We recommend using instead the
native Keras format, e.g. `model.save('my\_model.keras')` or
`keras.saving.save\_model(model, 'my\_model.keras')`.

Test Accuracy: 60.00%
Test Loss: 1.0054

Model saved to: /content/EcoSortAI/ecosort_model.h5

```

In [8]: # Cell 8: Load model (from local training or cloned GitHub repo) and
        # create prediction function

import os

MODEL_SAVE_PATH = '/content/EcoSortAI/ecosort_model.h5'

# if model isn't already saved locally (e.g. skipped training cells),
# grab it from the cloned GitHub repo instead
if not os.path.exists(MODEL_SAVE_PATH):
    repo_model_path = '/content/EcoSortAI_repo/ecosort_model.h5'
    if os.path.exists(repo_model_path):
        os.makedirs('/content/EcoSortAI', exist_ok=True)
        import shutil
        shutil.copy(repo_model_path, MODEL_SAVE_PATH)
        print("Model copied from cloned GitHub repo")
    else:
        print("WARNING: Model not found locally or in cloned repo. Run
        Cell 2 first.")

from tensorflow.keras.models import load_model
from tensorflow.keras.applications.mobilenet_v2 import preprocess_input
import cv2
import numpy as np

model = load_model(MODEL_SAVE_PATH)

CATEGORIES = ['organic', 'recyclable', 'hazardous', 'ewaste']
IMG_SIZE = 224

def predict_waste(img_path):
    img = cv2.imread(img_path)
    if img is None:
        print("Could not read image. Check the file path.")
        return None, None

    img = cv2.resize(img, (IMG_SIZE, IMG_SIZE))
    img = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
    img = np.expand_dims(img, axis=0)
    img = preprocess_input(img)

    prediction = model.predict(img, verbose=0)
    predicted_index = np.argmax(prediction[0])
    predicted_category = CATEGORIES[predicted_index]
    confidence = prediction[0][predicted_index] * 100

    return predicted_category, confidence

print("Prediction function ready")

```

WARNING:absl:Compiled the loaded model, but the compiled metrics have yet to be built. `model.compile\_metrics` will be empty until you train or evaluate the model.

Prediction function ready

```

In [10]: # Cell 9: Detailed recommendation engine with full guidance per category

disposal_info = {
    'organic': {
        'disposal': 'Compost Bin',
        'what_it_is': 'Biodegradable waste from food, plants, or natural
materials that decomposes naturally.',
        'how_to_dispose': 'Place in a compost bin or designated wet-
waste bin. Avoid mixing with plastic or metal.',
        'reuse_ideas': [
            'Home composting to create nutrient-rich soil for gardens',
            'Vermicomposting (using worms) for faster breakdown',
            'Biogas generation in community/municipal composting units'
        ],
        'where_to_recycle': 'Municipal wet-waste collection, home
compost pit, or community composting centers.',
        'environmental_note': 'Keeping organic waste out of landfills
reduces methane emissions, since decomposing organic matter in landfills
releases significant greenhouse gases.',
        'co2_saved': 0.10,
        'energy_saved': 1.5,
        'points': 5
    },
    'recyclable': {
        'disposal': 'Recycle Bin',
        'what_it_is': 'Materials like plastic, paper, glass, or metal
that can be reprocessed into new products.',
        'how_to_dispose': 'Rinse to remove food residue, then place in a
dry recyclable waste bin.',
        'reuse_ideas': [
            'Plastic bottles can be reprocessed into fibers for clothing
or new containers',
            'Paper and cardboard can be pulped into new paper products',
            'Metal cans can be melted down and reused indefinitely
without quality loss',
            'Glass can be crushed and remelted into new glass products'
        ],
        'where_to_recycle': 'Municipal recycling collection, local scrap
dealers, or authorized recycling facilities.',
        'environmental_note': 'Recycling reduces the need for virgin raw
material extraction, cutting down mining, deforestation, and
manufacturing emissions.',
        'co2_saved': 0.25,
        'energy_saved': 3.0,
        'points': 10
    },
    'hazardous': {
        'disposal': 'Hazardous Waste Collection Center',
        'what_it_is': 'Waste containing toxic, corrosive, or reactive
substances that can harm health and environment if mishandled.',
        'how_to_dispose': 'Never mix with regular household waste. Store
safely and hand over to an authorized hazardous waste collector.',
        'reuse_ideas': [
            'Batteries can have metals like lithium and cobalt recovered
and reused in new batteries',
            'Paint cans can be processed for material recovery at
specialized facilities',

```

```

        'Chemical containers should be professionally decontaminated
before any reuse'
    ],
    'where_to_recycle': 'Authorized hazardous waste collection
centers, battery drop-off points at electronics stores, or municipal
hazardous waste days.',
    'environmental_note': 'Improper disposal can contaminate soil
and groundwater for years. Proper handling prevents toxic leaching into
ecosystems.',
    'co2_saved': 0.05,
    'energy_saved': 0.5,
    'points': 15
},
'ewaste': {
    'disposal': 'Authorized E-Waste Recycling Center',
    'what_it_is': 'Discarded electronic devices and components
containing metals, plastics, and sometimes hazardous materials.',
    'how_to_dispose': 'Do not throw in regular trash. Wipe personal
data if applicable, then hand over to a certified e-waste recycler.',
    'reuse_ideas': [
        'Working parts can be refurbished and resold or donated',
        'Precious metals like gold, silver, and copper can be
extracted and reused',
        'Plastic casings can be shredded and reprocessed into new
plastic products'
    ],
    'where_to_recycle': 'Certified e-waste recyclers, manufacturer
take-back programs, or authorized e-waste collection drives.',
    'environmental_note': 'E-waste contains valuable recoverable
metals but also hazardous substances like lead and mercury if not
processed properly, making certified recycling essential.',
    'co2_saved': 0.50,
    'energy_saved': 5.0,
    'points': 15
}
}

def get_full_report(predicted_category, confidence):
    info = disposal_info[predicted_category]

    print(f"Predicted Category: {predicted_category.capitalize()}")
    print(f"Confidence: {confidence:.2f}%\n")

    print(f"What it is:\n{info['what_it_is']}\n")

    print(f"How to dispose:\n{info['how_to_dispose']}\n")

    print("Ways it can be reused/recycled:")
    for idea in info['reuse_ideas']:
        print(f"  - {idea}")
    print()

    print(f"Where to recycle:\n{info['where_to_recycle']}\n")

    print(f"Environmental impact:\n{info['environmental_note']}\n")

    print(f"CO2 Saved: {info['co2_saved']} kg")

```

```
print(f"Energy Saved: {info['energy_saved']} kWh")
print(f"Sustainability Points Earned: {info['points']}")

return info
```

```
In [12]: # Cell 10: Log prediction activity to CSV

import pandas as pd
import os
import datetime

CSV_LOG_PATH = '/content/EcoSortAI/activity_log.csv'

def log_activity(image_name, predicted_category, confidence, info):
    log_entry = {
        'Date': datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S'),
        'Filename': image_name,
        'Prediction': predicted_category,
        'Confidence': round(confidence, 2),
        'CO2_Saved': info['co2_saved'],
        'Score': info['points']
    }

    # if log file already exists, append to it, otherwise create new
    if os.path.exists(CSV_LOG_PATH):
        df = pd.read_csv(CSV_LOG_PATH)
        df = pd.concat([df, pd.DataFrame([log_entry])],
            ignore_index=True)
    else:
        df = pd.DataFrame([log_entry])

    df.to_csv(CSV_LOG_PATH, index=False)
    print("Activity logged successfully")
    return df

print("Logging function ready")
```

Logging function ready

```
In [13]: # Cell 11: Interactive multi-button interface for EcoSort AI

import ipywidgets as widgets
from IPython.display import display, clear_output
from google.colab import files
import pandas as pd
import matplotlib.pyplot as plt
from matplotlib.ticker import MaxNLocator
import os

MODEL_TEST_ACCURACY = 72.00

last_result = {'category': None, 'confidence': None, 'info': None}

upload_button = widgets.Button(description='Upload & Classify',
button_style='success', icon='upload')
disposal_button = widgets.Button(description='Disposal Guide',
button_style='info', icon='recycle')
reset_button = widgets.Button(description='Reset',
button_style='warning', icon='refresh')
dashboard_button = widgets.Button(description='Show Dashboard',
button_style='primary', icon='bar-chart')
clear_data_button = widgets.Button(description='Clear All Data',
button_style='danger', icon='trash')

output_area = widgets.Output()

def on_upload_clicked(b):
    with output_area:
        clear_output(wait=True)
        print("Opening file picker...\n")
        uploaded = files.upload()

        if not uploaded:
            print("No file selected.")
            return

        image_path = list(uploaded.keys())[0]
        predicted_category, confidence = predict_waste(image_path)

        if predicted_category is None:
            return

        info = disposal_info[predicted_category]

        last_result['category'] = predicted_category
        last_result['confidence'] = confidence
        last_result['info'] = info

        print(f"Predicted Category: {predicted_category.capitalize()}")
        print(f"Prediction Confidence: {confidence:.2f}%")
        print(f"Overall Model Accuracy: {MODEL_TEST_ACCURACY:.2f}%")

        log_activity(image_path, predicted_category, confidence, info)
        print("\nActivity logged. Click 'Disposal Guide' for detailed
instructions.")
```

```

def on_disposal_clicked(b):
    with output_area:
        clear_output(wait=True)
        if last_result['category'] is None:
            print("No image classified yet. Click 'Upload & Classify' first.")
        return

        info = last_result['info']
        category = last_result['category']

        print(f"DISPOSAL GUIDE – {category.capitalize()}\n")
        print(f"What it is:\n{info['what_it_is']}\n")
        print(f"How to dispose:\n{info['how_to_dispose']}\n")
        print("Ways it can be reused/recycled:")
        for idea in info['reuse_ideas']:
            print(f" - {idea}")
        print()
        print(f"Where to recycle:\n{info['where_to_recycle']}\n")
        print(f"Environmental impact:\n{info['environmental_note']}\n")

def on_reset_clicked(b):
    with output_area:
        clear_output(wait=True)
        print("Reset complete. Ready for a new image.")
        last_result['category'] = None
        last_result['confidence'] = None
        last_result['info'] = None

def on_dashboard_clicked(b):
    with output_area:
        clear_output(wait=True)

        if not os.path.exists(CSV_LOG_PATH) or
os.path.getsize(CSV_LOG_PATH) == 0:
            print("No activity data yet. Classify an image first.")
            return

        df = pd.read_csv(CSV_LOG_PATH)

        if df.empty:
            print("No activity data yet. Classify an image first.")
            return

        df['Date'] = pd.to_datetime(df['Date'])

        fig, axes = plt.subplots(2, 2, figsize=(15, 11))

        # Chart 1: Pie chart
        category_counts = df['Prediction'].value_counts()
        axes[0, 0].pie(category_counts, labels=category_counts.index,
autopct='%1.1f%%', startangle=90)
        axes[0, 0].set_title('Chart 1: Waste Category
Distribution\n(share of each category out of all items classified)',

```

```

fontsize=10)

    # Chart 2: Bar chart - daily classifications
    df['Day'] = df['Date'].dt.date
    daily_counts = df.groupby('Day').size()
    axes[0, 1].bar(daily_counts.index.astype(str),
daily_counts.values, color='seagreen')
    axes[0, 1].set_title('Chart 2: Daily Classifications',
fontsize=10)
    axes[0, 1].set_xlabel('Date')
    axes[0, 1].set_ylabel('Number of Images Classified')
    axes[0, 1].yaxis.set_major_locator(MaxNLocator(integer=True))
    axes[0, 1].tick_params(axis='x', rotation=45)

    # Chart 3: Line chart - cumulative score
    df['Cumulative_Score'] = df['Score'].cumsum()
    x_values = range(1, len(df) + 1)
    axes[1, 0].plot(x_values, df['Cumulative_Score'], marker='o',
color='darkorange')
    axes[1, 0].set_title('Chart 3: Sustainability Score
Trend\n(running total of points earned)', fontsize=10)
    axes[1, 0].set_xlabel('Classification Number (1st, 2nd, 3rd
image classified...)')
    axes[1, 0].set_ylabel('Cumulative Sustainability Score
(points)')
    axes[1, 0].xaxis.set_major_locator(MaxNLocator(integer=True))
    axes[1, 0].yaxis.set_major_locator(MaxNLocator(integer=True))

    # Chart 4: Bar chart - CO2 by category
    co2_by_category = df.groupby('Prediction')['CO2_Saved'].sum()
    axes[1, 1].bar(co2_by_category.index, co2_by_category.values,
color='steelblue')
    axes[1, 1].set_title('Chart 4: Total CO2 Saved by Category',
fontsize=10)
    axes[1, 1].set_xlabel('Waste Category')
    axes[1, 1].set_ylabel('Total CO2 Saved (kg)')

    plt.tight_layout()
    plt.show()

    print(f"\nTotal Sustainability Score: {df['Score'].sum()}
points")
    print(f"Total CO2 Saved: {df['CO2_Saved'].sum():.2f} kg")
    print(f"Total Images Classified: {len(df)}")

def on_clear_data_clicked(b):
    with output_area:
        clear_output(wait=True)
        empty_df = pd.DataFrame(columns=['Date', 'Filename',
'Prediction', 'Confidence', 'CO2_Saved', 'Score'])
        empty_df.to_csv(CSV_LOG_PATH, index=False)
        print("All activity data cleared. CSV log reset to empty.")
        last_result['category'] = None
        last_result['confidence'] = None
        last_result['info'] = None

```

```
upload_button.on_click(on_upload_clicked)
disposal_button.on_click(on_disposal_clicked)
reset_button.on_click(on_reset_clicked)
dashboard_button.on_click(on_dashboard_clicked)
clear_data_button.on_click(on_clear_data_clicked)

button_row = widgets.HBox([upload_button, disposal_button, reset_button,
dashboard_button, clear_data_button])
header = widgets.HTML("<h2> 🌱 EcoSort AI - Waste Classifier</h2>")

display(widgets.VBox([header, button_row, output_area]))
```

```
VBox(children=(HTML(value='<h2> 🌱 EcoSort AI - Waste
Classifier</h2>'), HBox(children=(Button(button_style='suc...
```

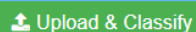
Exported with [runcell](#) — convert notebooks to HTML or PDF anytime at [runcell.dev](#).

EcoSort AI - Waste Classifier

Step 1: Upload an image with the control below.

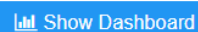
 Upload (1)

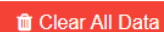
Step 2: Click Upload & Classify.

 Upload & Classify

 Disposal Guide

 Reset

 Show Dashboard

 Clear All Data

Select an image above, then click Upload & Classify.

Predicted Category: Hazardous
Prediction Confidence: 44.39%
Overall Model Accuracy: 72.00%
Activity logged successfully

Activity logged. Click 'Disposal Guide' for detailed instructions.

Step 1: Upload an image with the control below.

 Upload (1)

Step 2: Click Upload & Classify.

 Upload & Classify

 Disposal Guide

 Reset

 Show Dashboard

 Clear All Data

DISPOSAL GUIDE – Hazardous

What it is:

Waste containing toxic, corrosive, or reactive substances that can harm health and environment if mishandled.

How to dispose:

Never mix with regular household waste. Store safely and hand over to an authorized hazardous waste collector.

Ways it can be reused/recycled:

- Batteries can have metals like lithium and cobalt recovered and reused in new batteries
- Paint cans can be processed for material recovery at specialized facilities
- Chemical containers should be professionally decontaminated before any reuse

Where to recycle:

Authorized hazardous waste collection centers, battery drop-off points at electronics stores, or municipal hazardous waste collection events.

Environmental impact:

Improper disposal can contaminate soil and groundwater for years. Proper handling prevents toxic leaching into the environment.

